操作系统

线程与并发

周炽金

华东师范大学
软件工程学院

本章内容

上一章介绍了进程，本章讨论进程内的线程，以及多线程程序的编写。

- 一个进程里的线程，共享什么，又各自保存什么？
- 并发和并行有什么区别？增加 CPU 核数能提高多少性能？
- 用户线程与内核线程有哪些映射关系？
- 如何使用线程库、线程池和 OpenMP 编写多线程程序？
- 创建、等待、取消和退出线程时，需要注意什么？

资料：1. Operating System Concepts，第 4 章。

为什么需要多个线程?

只有一个线程时，一项操作阻塞就可能影响进程中的其他工作。

- 编辑器既要响应按键，也要检查拼写、读取文件。
- 服务器等待一个请求的数据时，还要接收其他请求。
- 把这些工作交给不同线程，各自保存执行状态。
- 线程复用进程已有的地址空间与资源，通常比新建进程开销小。
- 多线程也用于内核：不同任务可以等待不同事件。

资料：1. Operating System Concepts，第 4 章。

线程共享什么，各自保存什么?

同一进程中的线程共享资源，但各自的执行状态需要单独保存。

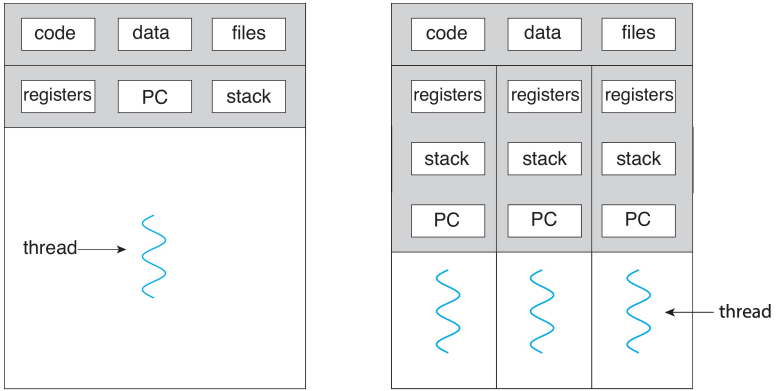
同一进程内通常共享	每个线程各自保存
代码、全局数据、堆	程序计数器 PC、寄存器
地址空间、打开的文件	栈、线程标识
进程的多种资源与属性	调度状态、线程局部数据

线程是执行流；进程为这些执行流提供资源和地址空间。

资料：1. Operating System Concepts，第 4 章；2. pthreads(7)。

单线程与多线程进程

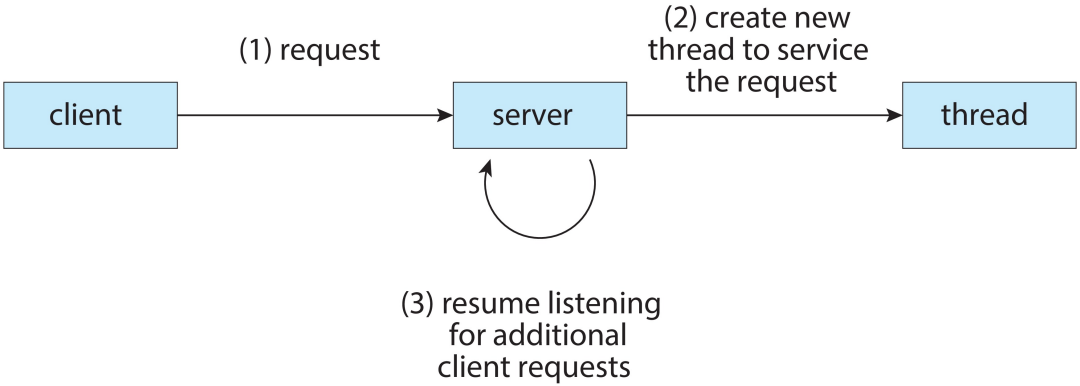
左图只有一个线程；右图中的多个线程共享代码、数据和文件，各有自己的执行状态。



资料：1. Operating System Concepts，第 4 章。

多线程服务器怎样处理请求?

以服务器为例：接收下一个请求，不必等当前请求处理完。



资料：1. Operating System Concepts，第 4 章。

多线程的优点

多线程除了便于处理多个请求，还可以改善响应速度、减少开销并利用多核 CPU。

- **响应性**：某个线程等待 I/O 时，界面仍有机会响应。
- **资源共享**：线程可直接访问共享数据和已打开的文件。
- **经济性**：通常减少创建执行流、切换地址空间的开销。
- **可扩展性**：可并行的任务能在多个 CPU 核上同时执行。

多个线程访问同一份可写数据时，需要同步。

资料：1. Operating System Concepts，第 4 章；2. pthreads(7)。

进程与线程的比较

多进程也能处理并发任务，两者在资源共享、开销和故障隔离方面有所不同。

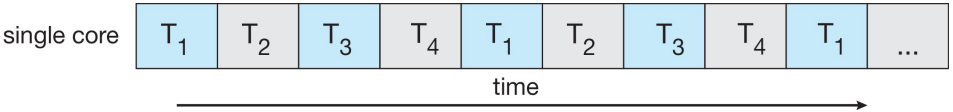
比较项	不同进程	同一进程的线程
资源组织	各自拥有资源，也可共享部分资源	复用进程资源
地址空间	通常彼此独立	共享同一地址空间
数据交换	IPC 或显式共享映射	可直接访问共享对象，仍需同步
创建与切换	通常开销较大	通常开销较小，并非零开销
故障影响	进程之间有隔离保护	错误可能破坏整个进程

资料：1. Operating System Concepts，第 4 章；2. pthreads(7)。

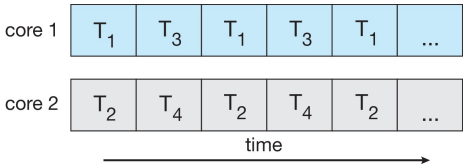
并发与并行

使用多个线程不一定意味着同时执行，需要区分并发与并行。

并发（Concurrency）：多个任务在一段时间内交替或同时执行。



并行（Parallelism）：多个任务在同一时刻实际执行。



资料：1. Operating System Concepts，第 4 章。

多核编程需要考虑的问题

要利用多个 CPU 核，需要合理划分任务，并处理好负载均衡和数据依赖。

- **任务划分**：图像处理中的哪些操作可以独立执行?
- **负载均衡**：是否有线程很早完成，其他线程仍在忙?
- **数据划分**：每个任务处理哪些像素、哪些图像?
- **数据依赖**：是否必须先得到其他任务的结果?
- **测试与调试**：不同调度顺序下，结果是否仍然正确?

资料：1. Operating System Concepts，第 4 章。

数据并行与任务并行

任务划分有两种常见方式：对不同数据执行相同操作，或分别执行不同操作。

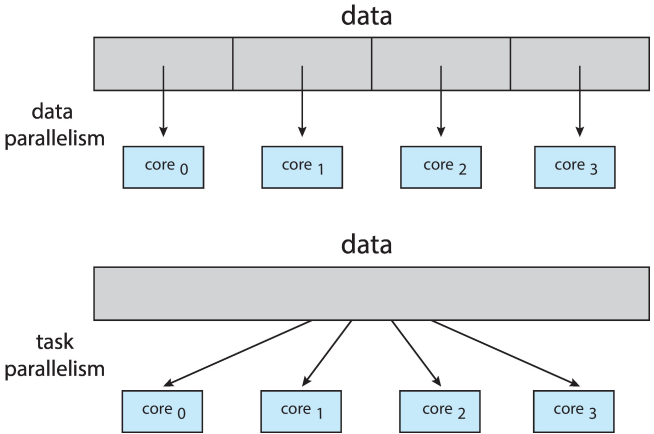
并行方式	划分方式	例子
数据并行	将数据集分为若干部分	多个线程分别调整一部分像素
任务并行	按功能或操作划分任务	不同任务分别负责解码、滤镜处理和编码

- 数据并行通常对不同数据做相同操作。
- 任务并行可以使用相同数据，也可以使用不同数据。
- 一个程序可以同时使用两种方式。

资料：1. Operating System Concepts，第 4 章。

数据并行与任务并行示意图

上图中，各 CPU 核执行相同操作；下图中，各 CPU 核执行不同操作。



资料：1. Operating System Concepts，第 4 章。

Amdahl 定律与加速比

即使部分计算可以并行，必须串行执行的部分仍然会限制整体加速比。

设固定工作量在单核上的执行时间为 T_1，串行比例为 S。

$$T_N = T_1 \left(S + \frac{1-S}{N} \right)$$

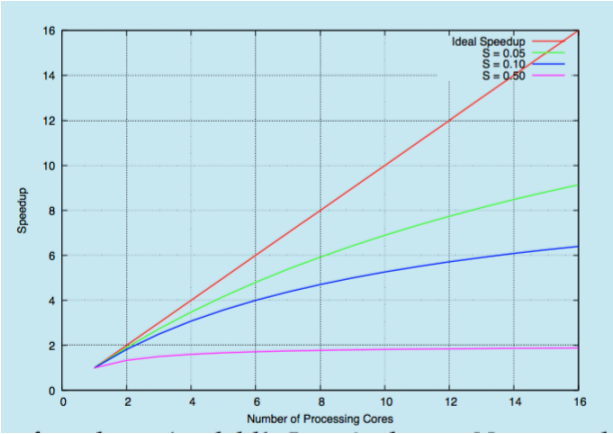
$$\text{Speedup}(N) = \frac{T_1}{T_N} = \frac{1}{S + (1-S)/N}$$

- N：参与计算的 CPU 核数；假设并行部分平均分配到各 CPU 核。
- $S > 0$ 时，$N \to \infty$，加速比趋近于 $1/S$。
- 若 $S = 0.25$、$N = 2$，理想加速比为 **1.6**。

资料：1. Operating System Concepts，第 4 章。

串行比例对加速比的影响

根据 Amdahl 定律，串行比例越大，增加 CPU 核数带来的收益越有限。



资料：1. Operating System Concepts，第 4 章。

100 个 CPU 核的理想加速比

以下比较使用 100 个 CPU 核时，不同串行比例对应的理想加速比。

串行比例 S	CPU 核数 N	理想加速比	CPU 核数趋于无穷时
0	100	100	无固定上限
0.10	100	9.174	10
0.25	100	3.883	4

增加 CPU 核数之前，先看耗时是否集中在不能并行的部分。

资料：1. Operating System Concepts，第 4 章。

Amdahl 定律的适用条件

Amdahl 定律采用理想化假设，实际程序还存在通信、同步和调度开销。

- 创建和调度线程、交换数据、等待锁都要时间。
- 内存带宽、缓存竞争和任务不均衡可能限制吞吐量。
- 模型固定问题规模；若数据量也增加，需要重新建立基准。
- 比较性能时保持工作量相同，并记录线程数和运行条件。

资料：1. Operating System Concepts，第 4 章。

用户级线程与内核级线程

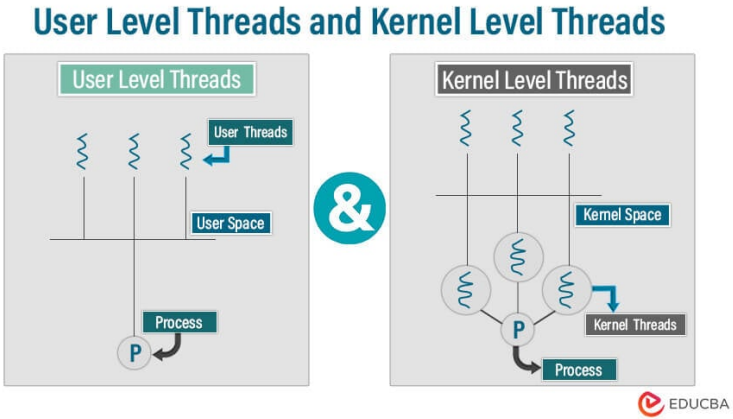
除了划分任务，多线程能否并行还取决于线程的实现方式和内核调度。

- **用户级线程**：由用户空间的线程库或运行时管理。
- **内核支持的线程**：内核保存状态，并把它作为可调度的执行实体。
- 用户线程通过与内核线程的映射，由内核调度到 CPU 上执行。
- Pthreads、Windows API、Java 提供的是接口；仅凭接口名称无法判断底层映射模型。
- Windows、Linux、macOS 等系统都支持内核调度的线程。

资料：1. Operating System Concepts，第 4 章；2. pthreads(7)；14. Microsoft：Processes and Threads。

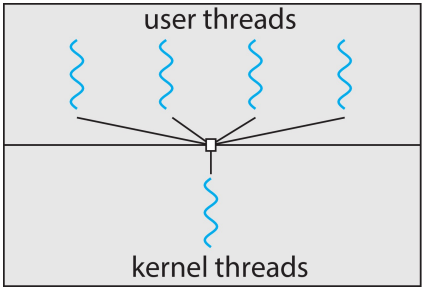
用户级线程与内核级线程示意图

用户级线程由线程库或运行时管理，内核级线程由操作系统管理和调度。



多对一模型

历史实现：GNU Pth。线程库在用户空间调度多个线程，但不能利用多个 CPU 核并行执行。



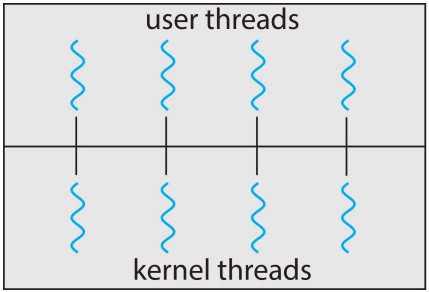
- 多个用户线程映射到一个内核线程。
- 运行时可以在用户空间切换执行流。
- 唯一内核线程因系统调用阻塞时，映射到它的其他用户线程也无法运行。
- 无法利用多个 CPU 核同时执行这些用户线程。

GNU Pth 用于事件驱动程序，采用协作式调度，线程主动让出执行机会。
示例：三个用户线程处理三个连接，共用一个内核线程；I/O 等待可由库协调。

资料：1. Operating System Concepts，第 4 章；27. GNU Pth。

一对一模型

主流原生线程实现：Linux NPTL、常规 Windows 线程都采用一对一模型。



user space

kernel space

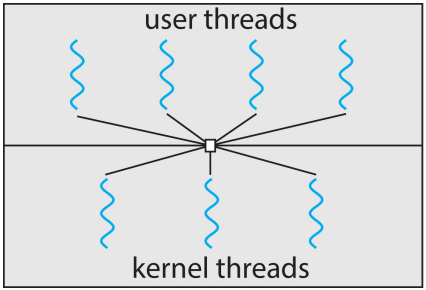
- 每个用户线程对应一个内核线程。
- 一个线程等待 I/O 时，其他可运行线程仍可被调度。
- 不同线程可以在多个 CPU 核上并行。
- 线程需要栈和内核管理资源，数量不能无限增加。

Linux 中，使用 NPTL 的 pthread_create 创建的每个线程，都有对应的内核调度实体。
例如服务器的工作线程：内核可分别调度它们；一对一不表示绑定某个 CPU 核。

资料：1. Operating System Concepts，第 4 章；2. pthreads(7)；14. Microsoft：Processes and Threads。

多对多模型

现代运行时中的应用：Go goroutine 和 Java 虚拟线程使用多对多调度。



user space

kernel space

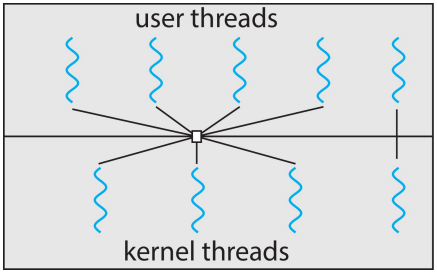
- 多个用户线程复用一组内核线程。
- 运行时将用户线程分配给内核线程，再由内核调度执行。
- 支持多核并行，同时控制内核线程的数量。
- 阻塞调用怎样处理，取决于运行时与系统的配合。

Go：多个 goroutine 由运行时调度到一组操作系统线程；goroutine 是轻量执行单元。
Java 21 起正式支持虚拟线程：多个虚拟线程由 JVM 调度到一组平台线程。
这与操作系统原生线程的一对一实现可以同时存在，讨论的是不同层次。

资料：1. Operating System Concepts，第 4 章；24. Microsoft：Fibers；28. Go：goroutine；29. Java 虚拟线程。

两级模型

历史实现：Solaris 8 默认线程库允许部分线程绑定，其余线程采用多对多映射。



- 在多对多模型上，允许某个用户线程绑定到内核线程。
- 图中部分用户线程共用内核线程，另一些与特定内核线程绑定。
- 这里绑定的是内核线程，不是固定某个 CPU 核。

Solaris 8：普通用户线程共用一组 LWP；THR_BOUND 可将指定线程绑定到专用 LWP。
LWP 是由内核调度的执行实体。例如 T1–T4 共用两个 LWP，T5 单独绑定第三个。
Solaris 9 起统一采用一对一实现；两级模型在这里作为历史设计介绍。

资料：1. Operating System Concepts，第 4 章；30. Solaris 8 线程库；31. Solaris 线程绑定。

线程库及其接口

了解线程的实现方式后，再看程序通过哪些接口创建和使用线程。

接口	常见操作	需要区分
POSIX Pthreads	create、join、同步	标准规定行为，库负责实现
Windows API	CreateThread、等待线程句柄	与 Windows 线程对象配合
Java Thread	start、join	JVM 管理，区分平台线程和虚拟线程

库可以在用户空间管理执行流，也可以调用内核提供的机制。

资料：2. pthreads(7)；18. Java：Thread；19. Microsoft：Creating Threads。

Pthreads：创建、执行、等待

Pthreads 提供线程创建、退出和等待等基本操作。

```
void *work(void *arg);
pthread_t tid;
int rc = pthread_create(&tid, NULL, work, &data);
if (rc != 0) { /* handle error code rc */ }
else { pthread_join(tid, NULL); }
```

- 新线程从 `work(arg)` 开始；创建者继续执行。
- 线程函数返回，或调用 `pthread_exit`，结束当前线程。
- `pthread_join` 等待指定线程结束，并可取得返回值。
- 多数 Pthreads 接口直接返回错误编号；完整例子检查每次调用。

资料：2. pthreads(7)；3. pthread_create(3)；4. pthread_join(3)。

两个线程执行同一个函数

下面的程序创建两个线程，调用同一个函数，分别传入不同的参数。

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
void* work(void* arg) {
    int id = *(int*)arg;
    for (int i = 0; i < 5; i++) {
        printf("Thread %d: iteration %d\n", id, i);
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2;

    pthread_create(&t1, NULL, work, &id1);
    pthread_create(&t2, NULL, work, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("All threads finished.\n");
    return 0;
}
```

资料：3. pthread_create(3)；4. pthread_join(3)。

同一程序的两次输出

运行刚才的程序：哪些输出次序必须保持，哪些可以变化？

```
chijin@LAPTOP-AAGNK9TF ~/w/os_course> ./pthread.out
Thread 1: iteration 0
Thread 2: iteration 0
Thread 1: iteration 1
Thread 2: iteration 1
Thread 1: iteration 2
Thread 2: iteration 2
Thread 2: iteration 3
Thread 1: iteration 3
Thread 2: iteration 4
Thread 1: iteration 4
All threads finished.
chijin@LAPTOP-AAGNK9TF ~/w/os_course> ./pthread.out
Thread 1: iteration 0
Thread 2: iteration 0
Thread 1: iteration 1
Thread 2: iteration 1
Thread 2: iteration 2
Thread 1: iteration 2
Thread 2: iteration 3
Thread 1: iteration 3
Thread 1: iteration 4
Thread 2: iteration 4
All threads finished.
```

资料：3. pthread_create(3)；4. pthread_join(3)。

创建线程后需要保证什么?

前面的程序中，两个线程各有参数，主线程等待它们结束；这涉及三个正确性要求。

参数有效

- 参数对象在工作线程使用期间必须有效；不要把反复修改的循环变量地址传给多个线程。

访问同步

- 多个线程修改同一个变量时需要同步。例如 `count++` 包含读、改、写，不能保证原子性。

等待完成

- 使用 `join` 等待线程结束，不能用 `sleep` 判断是否完成；`join` 也不能保证执行期间的共享访问互斥。

资料：2. pthreads(7)；3. pthread_create(3)；4. pthread_join(3)。

Java：start() 与 run() 的区别

Java 使用 start() 启动线程，使用 join() 等待线程结束。

```
Thread t = new Thread(() -> {
    System.out.println("worker");
});
t.start();
System.out.println("main");
t.join();
System.out.println("done");
```

- `start()` 启动新的执行流；直接调用 `run()` 是普通方法调用。
- `join()` 等待线程终止；done 在 worker 和 main 都打印之后输出。
- 此例是平台线程；Java 也提供由运行时调度的虚拟线程。

资料：18. Java：Thread。

Windows：线程创建与等待

Windows 通过线程句柄标识线程，并提供等待线程结束的接口。

```
DWORD WINAPI work(LPVOID arg) { return 0; }
HANDLE h = CreateThread(NULL, 0, work, NULL, 0, NULL);
if (h == NULL) { /* handle GetLastError() */ }
else {
    WaitForSingleObject(h, INFINITE);
    CloseHandle(h);
}
```

- 线程句柄可用于等待线程结束。
- 关闭句柄释放该句柄，不等于终止仍在运行的线程。
- 完整例子还检查等待结果，并读取线程退出码。

资料：19. Microsoft：Creating Threads。

线程接口之后：任务管理与并行编程

前面介绍了怎样创建、启动和等待线程。接下来讨论任务较多时，怎样管理线程并组织计算。

- **线程池**：复用工作线程，控制线程数量，处理不断到来的请求。
- **Fork–Join**：把一个计算任务分成子任务，等待并合并结果。
- **OpenMP**：通过编译指令实现并行区域、循环和归约。
- **GCD**：把任务提交到队列，由系统安排执行。

这些方法减少线程管理工作，但程序仍需处理数据共享和任务依赖。

资料：1. Operating System Concepts，第 4 章。

显式线程与隐式线程

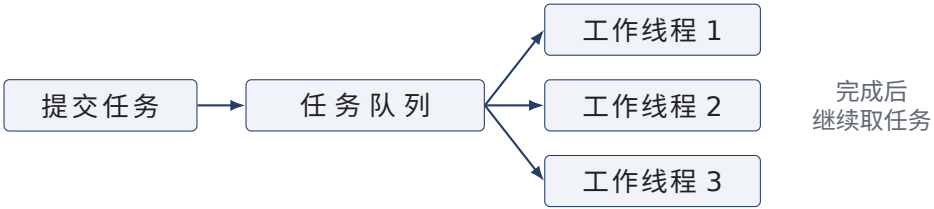
回到服务器：请求不断到来时，逐个创建和回收线程会增加管理成本。

- 显式线程：程序负责创建、传参、等待和清理线程。
- 隐式线程：程序描述任务或并行区域，运行时安排执行。
- **线程池**：把多个任务交给一组可复用线程。
- **Fork–Join**：拆分工作，再等待子任务并合并结果。
- **OpenMP / GCD**：用指令或队列表达待执行的工作。

资料：1. Operating System Concepts，第 4 章。

线程池的工作原理

请求先进入任务队列，工作线程从队列取出任务，完成后继续处理下一个。



- 工作线程完成任务后，继续取下一项任务。
- 复用线程可减少重复创建的开销，并限制工作线程数量。
- 任务队列长度也要管理，不能把线程上限当成内存上限。
- 定时任务可由调度器提交给线程池；线程池不一定自带定时功能。

资料：1. Operating System Concepts，第 4 章；5. Python：concurrent.futures。

Python：提交六个任务

下面用 Python 线程池提交六个任务，最多使用三个工作线程。

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def work(task_id):
    print(f"Task {task_id} is running")
    time.sleep(1)
    print(f"Task {task_id} done")

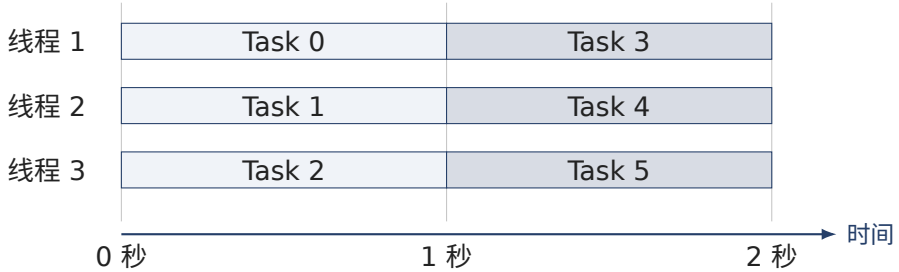
with ThreadPoolExecutor(max_workers=3) as pool:
    tasks = []
    for i in range(0,6):
        task = pool.submit(work, i)
        tasks.append(task)
    for f in as_completed(tasks):
        f.result()

print("All tasks finished.")
```

资料：5. Python：concurrent.futures；6. Python：threading。

三个线程怎样处理六个任务？

下图表示六个任务的执行过程，同一个工作线程可以先后处理多个任务。



- 每个任务用 `sleep(1)` 模拟约 1 秒等待。
- 三个线程的等待时间可以重叠，六个任务约分两轮完成。
- 某个线程先空闲，就能取下一项；不需要等这一批任务全部完成。

资料：5. Python：concurrent.futures；6. Python：threading。

Python 多线程：等待任务与计算任务

前例约两秒完成六次一秒等待，是因为等待时间重叠；计算任务的情况不同。

任务主要在做什么	多线程能起什么作用
等待网络、文件，或 sleep	等待时间可以重叠，缩短所有任务完成所需的时间
执行纯 Python 计算	启用 GIL 的 CPython 中，同一解释器通常不能并行执行 Python 字节码

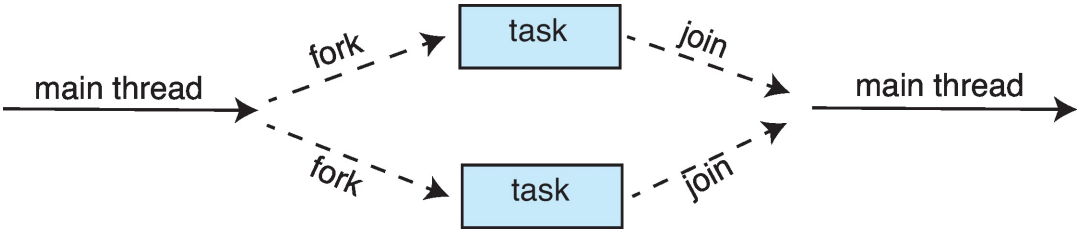
- **GIL（全局解释器锁）**通常只允许同一解释器的一个线程执行 Python 字节码。
- 因此，线程池能让等待任务并发进行，不代表纯 Python 计算能用上多个 CPU 核。
- 计算密集型任务可考虑多进程；是否加速，应在相同工作量下测量。

原生扩展释放 GIL，或使用禁用 GIL 的自由线程版本时，需要另外分析。

资料：5. Python：concurrent.futures；6. Python：threading。

Fork–Join 并行模型

线程池负责分配执行任务的线程；Fork–Join 描述任务的拆分、等待和结果合并。



资料：1. Operating System Concepts，第 4 章。

Fork–Join 伪代码

问题较小时直接求解，否则分成子问题，等待子问题完成后合并结果。

```
Task(problem)
   if problem is small enough
      solve the problem directly
   else
      subtask1 = fork(new Task(subset of problem)
      subtask2 = fork(new Task(subset of problem)

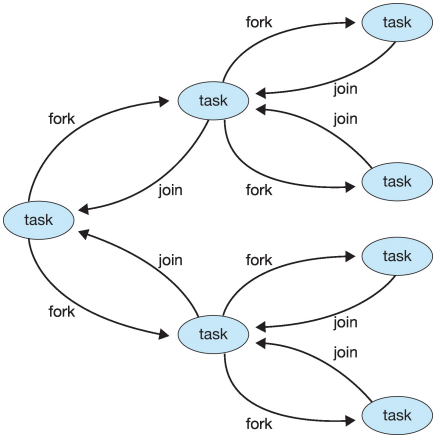
      result1 = join(subtask1)
      result2 = join(subtask2)

      return combined results
```

资料：1. Operating System Concepts，第 4 章。

Fork–Join 任务树

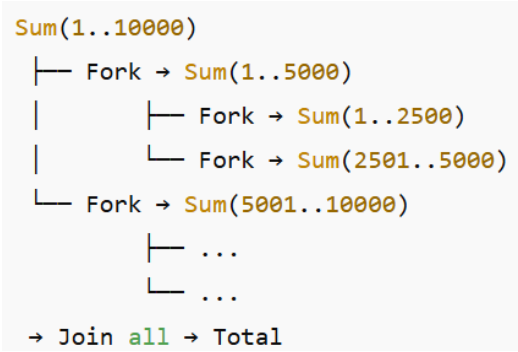
递归划分的任务形成一棵树，每个父任务都需要等待子任务完成。



资料：1. Operating System Concepts，第 4 章。

分治求和：1 到 10000

将求和区间分为左右两段，分别计算后相加，也可以对子区间继续划分。



- 左右区间分别求和，返回后把两个部分相加。
- 小区间直接计算，避免为几个加法创建线程。
- 任务可以递归拆分，工作线程的数量则单独限制。

资料：1. Operating System Concepts，第 4 章。

递归求和：限制线程创建数量

```
def fork_join_sum(data, depth=0):
    if len(data) <= 1024 or depth >= 2:
        return sum(data)
    mid = len(data) // 2
    box = [None]
    def left():
        box[0] = fork_join_sum(data[:mid], depth + 1)
    t = Thread(target=left)
    t.start()
    right = fork_join_sum(data[mid:], depth + 1)
    t.join()
    return box[0] + right
```

当前线程计算右半段，新线程计算左半段；等待新线程结束后再相加。

资料：6. Python：threading。

任务粒度与线程数量

长度阈值决定任务大小，递归深度上限限制创建的线程数量。

- 长度阈值决定何时改为直接计算。
- 深度上限为 2 时，最多同时有 4 条执行流，包括主线程。
- 两个子任务分别保存结果，完成后再合并，避免同时更新同一个累加器。
- 不把递归任务直接改成“向同一小线程池提交，再在池内等待”。

所有工作线程若都在等队列里的子任务，可能无人执行这些子任务。

资料：5. Python：concurrent.futures；6. Python：threading。

OpenMP：并行区域

OpenMP 通过编译指令指定并行区域，由编译器和运行时负责线程管理。

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
   /* sequential code */

   #pragma omp parallel
   {
      printf("I am a parallel region.");
   }

   /* sequential code */

   return 0;
}
```

- 为 C、C++、Fortran 提供编译指令与运行时接口。
- `parallel` 创建一个线程组，各线程执行并行区域中的代码。
- 线程数由环境变量、子句及运行时限制共同决定。
- 所有线程完成并行区域后，继续执行后面的串行代码。

资料：7. OpenMP 5.2。

OpenMP：并行 for 循环

处理数组时，可以用 parallel for 将不同循环迭代分配给不同线程。

```c
#include <stdio.h>
#include <omp.h>
#include <unistd.h>
int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int b[5] = {10, 20, 30, 40, 50};
    int c[5];

    #pragma omp parallel for
    for (int i = 0; i < 5; i++) {
        c[i] = a[i] + b[i];
        sleep(1);
        printf("%d\n", c[i]);
    }
    return 0;
}
```

资料：7. OpenMP 5.2。

并行循环的结果与输出顺序

各次迭代分别写入不同的数组元素，计算结果不受执行顺序影响。

```
chijin@LAPTOP-AAGNK9TF ~/w/o/3_thread> clang++ openmp.cc -o openmp.out -fopenmp
chijin@LAPTOP-AAGNK9TF ~/w/o/3_thread> ./openmp.out
11
33
22
55
44
chijin@LAPTOP-AAGNK9TF ~/w/o/3_thread> ./openmp.out
55
44
22
11
33
```

资料：7. OpenMP 5.2。

OpenMP：归约求和

求和时，各线程先分别累加，再合并结果，避免同时修改同一个变量。

```
long long total = 0;
#pragma omp parallel for reduction(+:total)
for (int i = 1; i <= 10000; ++i)
    total += i;
printf("total=%lld\n", total);
```

- `reduction` 让各线程使用私有变量累加，最后合并各线程的结果。
- 直接让所有线程无同步地修改同一个 `total` 会产生数据竞争。
- 整数求和例子结果为 50005000。

资料：7. OpenMP 5.2。

GCD：任务队列

GCD 提供任务队列接口，由系统负责安排任务的执行。

```
dispatch_queue_t q = dispatch_get_global_queue(
    DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);
dispatch_group_async(group, q, ^{ work(1); });
dispatch_group_async(group, q, ^{ work(2); });
dispatch_group_wait(group, DISPATCH_TIME_FOREVER);
```

- 提交任务后，由系统选择线程执行。
- 串行队列一次执行一个任务；并发队列允许多个任务并发执行。
- 队列和线程不是一一对应的；组用于等待一组任务。

资料：20. Apple：Dispatch Queues。

多线程程序中的管理问题

前面讨论了任务如何执行。接下来讨论进程和线程在运行过程中遇到的管理问题。

- **进程复制与程序替换**：多线程进程调用 fork、exec 后会发生什么？
- **信号处理**：进程收到通知时，由哪个线程接收和处理？
- **线程取消与资源回收**：任务不再需要时，怎样结束线程并释放资源？
- **线程局部存储**：哪些数据需要由每个线程单独保存？

资料：1. Operating System Concepts，第 4 章。

多线程进程中的 fork 与 exec

多线程程序还需要考虑进程复制、信号处理和线程退出等问题。

- `fork()` 后，子进程只保留调用它的那个线程。
- 地址空间会被复制，其他线程留下的锁状态也可能被复制。
- 其他线程不会在子进程中继续运行，原先持锁者可能已不存在。
- 成功执行 `exec` 会替换程序映像，其他线程被终止。
- 子进程在 `exec` 前的操作应满足异步信号安全要求。

资料：8. fork(2)；9. execve(2)；23. signal-safety(7)。

信号的产生与处理

程序运行时，信号用于通知进程或线程发生了某种事件。

- 某个事件产生信号，系统将它递送到相应进程或线程。
- **同步信号**由当前执行触发；**异步信号**可由进程外部事件产生。
- 默认处置可能是终止、停止、继续或忽略。
- 程序可以安装处理函数，也可以对部分信号选择忽略。
- `SIGKILL`、`SIGSTOP` 不能被捕获、阻塞或忽略。
- 处理函数可能打断原执行流程，只能做符合要求的操作。

资料：10. signal(7)；23. signal-safety(7)。

多线程进程中，哪个线程接收信号？

多线程进程收到信号后，需要按照信号类型和各线程的设置确定接收线程。

情形	接收规则
线程定向信号、线程触发的同步故障	指定线程或引发故障的线程
进程定向信号	由一个符合接收条件的线程接收
统一处理控制信号	其他线程阻塞该信号，专门线程调用 sigwait

- 信号处置通常属于进程；信号屏蔽字属于线程。
- 给进程发送一次信号，不等于向每个线程广播一次。

资料：10. signal(7)；21. pthread_sigmask(3)。

线程取消

当任务不再需要继续执行时，可以请求取消相应线程。

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

 . . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid,NULL);
```

- **异步取消**：允许在任意位置响应，资源清理很困难。
- **延迟取消**：到取消点才响应，便于安排清理。
- `pthread_cancel` 请求取消；何时生效由目标线程的状态和类型决定。
- 最后 `join` 等待线程退出，并取得退出结果。

资料：11. pthread_cancel(3)；12. pthread_setcancelstate(3)。

取消状态、取消类型与取消点

取消状态决定是否允许取消，取消类型决定何时响应取消请求。

是否允许取消	取消类型	收到请求后
Disabled	任意	保留请求，暂不响应，继续执行
Enabled	Deferred	到达取消点时响应
Enabled	Asynchronous	可以在任意位置响应

- 新线程默认允许取消，类型为延迟取消。
- `pthread_testcancel()` 显式检查；部分阻塞接口也是取消点。
- 响应取消时，执行已注册的清理处理函数，再进行线程退出清理。

资料：11. pthread_cancel(3)；12. pthread_setcancelstate(3)。

线程退出与资源回收

线程退出时，还需要释放内存、锁等应用资源，并回收线程自身的资源。

- 明确哪些资源属于任务，哪些资源由线程长期持有。
- 用停止标志请求协作退出时，标志本身也需要正确同步。
- 延迟取消可配合 `pthread_cleanup_push/pop` 注册清理动作。
- 退出后由 `join` 回收可等待（joinable）线程的资源；分离线程按规则自动回收。

停止计算、释放应用资源、回收线程资源，是相关但不同的步骤。

资料：2. pthreads(7)；4. pthread_join(3)；11. pthread_cancel(3)。

线程局部存储（TLS）

有些数据需要在线程的多次函数调用之间保留，但希望每个线程单独保存一份。

```c
__thread int counter = 0;  // 每个线程都有自己的 counter（TLS 变量）

void* work(void* arg) {
    for (int i = 0; i < 3; i++) {
        counter++;
        printf("Thread %ld: counter = %d\n", (long)arg, counter);
    }
    return NULL;
}
```

- **TLS（Thread-local storage）**：同一变量名，每个线程有自己的实例。
- 同一线程多次调用函数，可以继续使用自己的 TLS 数据。
- 适合线程内的缓存、错误信息和上下文。
- 在线程池中，TLS 随工作线程保留，不会自动随任务重置。

资料：2. pthreads(7)；13. pthread_key_create(3)。

局部变量、static 与 TLS

普通局部变量、static 变量和 TLS 变量的存储期及共享范围不同。

声明方式	实例与生命周期	多线程使用时
普通自动局部变量	每次调用时创建，离开作用域后销毁	通常位于当前线程栈上
普通 static 变量	程序运行期间保留同一实例	同一进程线程可共享
_Thread_local 变量	每个线程一份，在线程退出时销毁	同一线程可跨调用使用

Pthreads 还提供 key 与线程特定数据接口，可注册退出时的析构函数。

资料：2. pthreads(7)；13. pthread_key_create(3)。

操作系统如何实现线程

线程各自保存执行状态，并与其他线程共享进程资源。最后看这些关系在实际系统中怎样表示。

- **Windows**：线程对象、调度信息和用户空间的线程环境。
- **Linux**：task_struct、线程组，以及 clone 指定的资源共享关系。
- **实际观察**：通过系统命令查看一个进程中的线程。

结合这些实现，再讨论浏览器使用多进程或多线程的取舍。

资料：1. Operating System Concepts，第 4 章。

Windows 的线程管理

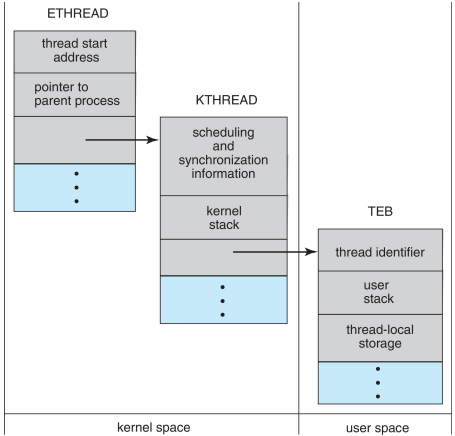
以 Windows 为例，查看线程的执行状态、调度信息和 TLS 分别由哪些结构保存。

- 常规 Windows 线程采用一对一映射。
- 每个线程有标识、寄存器现场、用户栈、内核栈及线程局部信息。
- **ETHREAD**：内核中的线程对象与管理信息。
- **KTHREAD**：调度、同步等低层线程信息。
- **TEB**：用户空间中的线程环境信息，包括 TLS 等。

资料：1. Operating System Concepts，第 4 章；14. Microsoft：Processes and Threads；15. Microsoft：Kernel Structures；16. Microsoft：TEB。

Windows 线程的数据结构

图中展示了 Windows 线程的主要数据结构及其在用户空间、内核空间的位置。



资料：1. Operating System Concepts，第 4 章；15. Microsoft：Kernel Structures；16. Microsoft：TEB。

Linux 的线程管理

Linux 为每个线程保存独立的任务结构，通过资源共享关系表示线程之间的联系。

- Linux 用 `task_struct` 管理任务；同一进程的各线程也各有任务结构。
- 创建时可通过 `clone/clone3` 的标志选择共享哪些资源。
- 同一线程组的线程具有不同 TID，通常共享 TGID。
- `pthread_create` 由库负责分配栈、初始化 TLS，并请求内核创建线程。
- 用户程序一般使用线程库，不直接用 clone 实现完整的线程创建过程。

资料：2. pthreads(7)；17. clone(2)。

clone 标志与资源共享

clone 的标志指定新任务与创建者共享哪些资源，以及是否加入同一线程组。

标志	作用
CLONE_VM	虚拟地址空间
CLONE_FILES	文件描述符表
CLONE_FS	当前目录、根目录等文件系统上下文
CLONE_SIGHAND	信号处置表
CLONE_THREAD	加入同一线程组

共享信号处置表，不等于共享各线程的信号屏蔽字。

资料：17. clone(2)。

在 Linux 中查看一个进程的线程

Linux 提供命令和 /proc 接口，可以查看一个进程中的线程。

```
ps -L -p <PID> -o pid,tid,stat,comm
ls /proc/<PID>/task
cat /proc/<PID>/status
```

- `PID` 换成目标进程编号；观察主线程和工作线程的 TID。
- `/proc/<PID>/task/` 下每个子目录对应一个线程。
- `status` 中的 `Threads` 显示读取时的线程数量。
- 线程可能随时创建或结束，两次读取结果不一定相同。

资料：25. proc_pid_task(5)；26. proc_pid_status(5)。

课堂讨论：如果浏览器只用一个进程

按上一章“每个标签页一个进程”的简化模型讨论：

Google Chrome 浏览器对于每一个页面、每一个插件，都分别用一个进程来进行渲染。如果 Chrome 设计成为只有一个进程，但是每一个页面和插件都用单独的线程来处理，会有什么好处？会有什么坏处？

资料：22. Chromium：Multi-process Architecture。

本章小结

本章介绍了线程的基本概念、实现方式、编程接口，以及协作和退出时的问题。

- **进程与线程**：资源和地址空间由进程组织，线程各自执行程序。
- **并发与并行**：交替执行不等于同时执行。
- **映射模型**：描述用户线程与内核线程的对应关系。
- **任务与线程**：一个工作线程可依次执行多个任务。
- **共享与生命周期**：既要协调数据访问，也要安排等待和退出。

资料：1. Operating System Concepts，第 4 章。

课堂练习

以下练习涉及资源共享、并行加速、线程调度、TLS 和线程取消。

- 一个线程有自己的栈，是否意味着其他线程绝对不能访问它？
- 串行部分占 20%，使用 4 个 CPU 核时理想加速比是多少？
- 一对一模型中的一个线程等待 I/O，是否会让整个进程都无法运行？
- 在线程池中用 TLS 保存请求编号，下一个任务开始时需要注意什么？
- `pthread_cancel` 成功返回后，目标线程是否一定已经结束？

参考资料

[1] Silberschatz, Galvin, Gagne (2018) —Operating System Concepts, 10th ed., Chapter 4: Threads & Concurrency

[2] Linux man-pages —pthreads(7): POSIX threads

[3] Linux man-pages —pthread_create(3): create a new thread

[4] Linux man-pages —pthread_join(3): join with a terminated thread

[5] Python Documentation —concurrent.futures: Launching parallel tasks

[6] Python Documentation —threading: Thread-based parallelism

[7] OpenMP ARB (2021) —OpenMP API Specification, Version 5.2

[8] Linux man-pages —fork(2): create a child process

[9] Linux man-pages —execve(2): execute program

[10] Linux man-pages —signal(7): overview of signals

[11] Linux man-pages —pthread_cancel(3): send a cancellation request to a thread

[12] Linux man-pages —pthread_setcancelstate(3): set cancelability state and type

[13] Linux man-pages —pthread_key_create(3): create a key for thread-specific data

[14] Microsoft Learn —About Processes and Threads

[15] Microsoft Learn —Windows kernel opaque structures

[16] Microsoft Learn —TEB structure (winternl.h)

[17] Linux man-pages —clone(2): create a child process

[18] Oracle —Java SE 25 API: java.lang.Thread

[19] Microsoft Learn —Creating Threads

[20] Apple —Concurrency Programming Guide: Dispatch Queues

[21] Linux man-pages —pthread_sigmask(3): examine and change mask of blocked signals

[22] The Chromium Projects —Multi-process Architecture

[23] Linux man-pages —signal-safety(7): async-signal-safe functions

[24] Microsoft Learn —Fibers

[25] Linux man-pages —proc_pid_task(5): thread information

[26] Linux man-pages —proc_pid_status(5): memory usage and status information

[27] GNU —GNU Pth: The GNU Portable Threads

[28] The Go Programming Language —FAQ: Why goroutines instead of threads?

[29] OpenJDK —JEP 444: Virtual Threads

[30] Oracle —Solaris 8: Linking With the Alternate One-level libthread Library

[31] Oracle —Syntax of Oracle Solaris Threads Functions